

NKU.png

南 开 大 学

计 算 机 学 院

并行程序设计期末实验报告

标题标题标题

飏

年级：2017 级

专业：计算机科学与技术

指导教师：王刚

2026 年 3 月 15 日

目录

- 一、 绪论 1
- 二、 相关工作 1
 - (一) 推荐系统研究现状 1
 - 1. 传统推荐算法的原理与局限 1
 - 2. 基于大语言模型的推荐研究进展 2
 - 3. 现有研究的空白 2
 - (二) 多 Agent 系统研究 2
 - 1. Agent 的基本概念与理论基础 2
 - 2. 多 Agent 系统的分布式协作机制 3
 - 3. 多 Agent 架构在推荐系统中的应用潜力 3
 - (三) 可视化辅助的 LLM 人机交互 3
 - 1. 推理过程的可解释性可视化 4
 - 2. 多维推荐结果的可视化比较 4
 - 3. 对话交互中的视觉引导机制 4
- 三、 需求分析与方法设计 5
 - (一) 需求层面分析 5
 - 1. 调研设计与数据来源 5
 - 2. 用户决策痛点分析 5
 - 3. 系统功能需求归纳 5
 - (二) 系统总体框架设计 6
 - 1. 系统整体运行流程 6
 - 2. 四层模块架构设计 6
 - 3. 动态领域扩展机制 6
 - (三) 本章小结 6
- 四、 核心算法设计与系统设计 7
 - (一) 用户需求语义解析与参数提取 7
 - (二) 异构商品多维评分模型 8
 - (三) 双层场景化推荐引擎 9
 - (四) 跨品类生态推荐算法 10
 - (五) 本章总结 11
- 五、 系统架构实现与代码落地 11
 - (一) 全栈基础架构与运行环境实现 11
 - (二) 多 Agent 推荐模块实现 12
 - (三) 商品数据层实现与数据清洗 14
 - (四) 用户界面与交互式可视化实现 15
 - (五) 本章小结 16

六、 系统评估与真实场景化分析	16
(一) 评估方案设计与测试基准	16
(二) 核心功能逻辑验证	17
(三) 系统效能评估分析	17
(四) 从输入到图表的终局案例演示	17
1. 案例 1: 大学生的轻薄办公场景 (笔记本电脑)	18
2. 案例 2: 极致性能的游戏发烧场景 (全场景生态组合)	18
3. 案例 3: 追求性价比的摄影入门场景 (相机)	18
(五) 本章小结	18
七、 总结与展望	18
(一) 本文贡献与总结	18
1. 提出面向消费场景的对话式推荐范式	18
2. 构建兼顾灵活性与可靠性的双层推荐架构	18
3. 实现跨品类协同推荐	19
4. 以可视化手段增强推荐的可解释性	19
(二) 局限性与改进方向	19
1. 商品数据的实时性不足	19
2. 复杂语义场景下的解析局限	19
3. 个体偏好的持续建模缺失	20
4. 后续改进方向	20

一、绪论

随着电子商务的快速发展,网络购物已深度融入人们的日常生活。商品数量的爆炸式增长虽然带来了更丰富的选择,却也让消费者在面对庞大的商品列表、繁杂的规格参数以及质量不一的用户评价时,常常难以迅速判断哪些商品真正符合自己的需求。研究显示,当信息量过大时,用户的认知负担会显著增加,决策效率随之下降,甚至可能出现决策回避或购买后的懊悔情绪。换句话说,信息越多,决策反而越难。

现有的推荐方式难以破解这一困境。传统协同过滤更依赖历史行为,对用户当下的即时意图理解不足,并常受到冷启动限制;基于大语言模型的纯对话式推荐虽然能理解自然语言,但容易生成不准确或无法验证的商品信息;内容型测评平台信息丰富,却依赖创作者的主观经验,用户仍需投入大量时间自行阅读和归纳。这些问题共同导致一个现实:目前的工具都无法同时满足“理解即时需求、提供可靠事实依据、以可视化方式辅助决策”这三项关键能力,也就难以真正帮用户在海量信息中高效做出选择。

在上述背景下,本文提出并实现了一套基于多 Agent 架构的智能商品选购推荐系统,通过把模型对语言的理解力,与结构化商品库中明确、可靠的数据结合起来,再利用三级分工的 Agent 路由体系,让用户一句自然语言的需求,能够被准确拆解、匹配,并最终清晰直观的可视化方式呈现推荐结果。系统支持 10 大数码品类的精准推荐与未知品类的动态扩展,给出三个推荐商品并附带推荐理由和优缺点分析,此外通过多维雷达图与分组柱状图将推荐商品进行可视化的比较,有效降低用户决策门槛。

各章概述

第二章对推荐系统、多 Agent 技术以及可解释性人机交互的相关研究进行综述;第三章依据问卷调查结果开展需求分析,并给出包含数据采集流程在内的系统总体架构;第四章围绕双层推荐引擎、多维度评分机制和生态化推荐方法等核心算法展开说明;第五章介绍系统的具体工程实现;第六章通过功能测试与用户评价对系统进行验证;第七章对全文工作进行总结,并提出后续可能的优化方向。

二、相关工作

(一) 推荐系统研究现状

1. 传统推荐算法的原理与局限

现有的推荐方法大致可以分为三类,各有其形成背景与适用范围。

协同过滤 (Collaborative Filtering, CF) 是推荐系统中最早成型、也最具代表性的一类方法。无论是基于用户还是基于物品的实现,它们都建立在同一个朴素但有效的假设上:相似的用户会喜欢相似的物品 [16]。矩阵分解方法的引入显著提升了协同过滤在大规模数据上的表现, SVD++ 等模型在工业实践中得到广泛应用 [12], 让这一路线在工业界的实践相对成熟,尤其是在数据规模足够大、用户偏好稳定时,往往能给出相当可靠的结果。然而,这类方法本质上依赖历史交互,因此在数据稀疏的情况下往往力不从心;当新用户或新商品刚进入系统、缺乏足够行为记录时,“冷启动”问题会格外明显 [1]。此外,协同过滤并不善于理解用户当下的即时意图。例如,两位用户都可能浏览过同一类笔记本电脑,但其中一位正在寻找轻薄便携款,另一位关注的是游戏性能,这类需求差异不会自然地体现在历史行为里,也超出了协同过滤的辨别能力。

基于内容的过滤 (Content-Based Filtering) 试图绕开对其他用户行为的依赖,通过分析物品属性和用户过往偏好建立匹配关系 [14], 因此在用户规模较小时也能保持稳定性能。它的挑战在于对特征工程的高度依赖:人工构建的特征往往难以覆盖商品的全部关键要素,且维护成本

高；系统很容易陷入“越推越窄”的过滤气泡，难以拓展用户的潜在兴趣。此外，面对自然语言中隐含的偏好描述，例如“想要一个适合旅行用、充电久一点的相机”，传统内容过滤缺乏足够的语义理解能力，往往无法有效处理，因此常被质疑“听不懂人话”。

基于知识的推荐（Knowledge-Based Recommendation）则通过领域规则或知识图谱来约束匹配空间，在如数码产品、房产、汽车等高价值、低频决策场景中表现尤为稳健 [6] [7]。它的优势在于决策链条清晰、可解释性强，可以明确指出某个推荐结果背后的逻辑依据。但这类方法的构建门槛较高：规则需要密集的领域知识支撑，维护难度也随商品更新而增加；同时，系统面对开放式自然语言的表达缺乏弹性，扩展性相对不足。总体而言，它适合结构化、逻辑性明确的任务，却难以应对用户真实咨询场景中那种复杂、多变、带有模糊需求的对话式偏好表述。

2. 基于大语言模型的推荐研究进展

近年来，大语言模型（LLM）的出现为推荐系统带来了一条全新的技术路线。以 TALLRec、LLM4Rec 等模型为代表的一系列研究尝试将 LLM 置于推荐流程的“语义中枢”位置 [4] [23]，让用户以自然语言表达的需求不再需要被手工拆解成标签或关键词，而是能够直接转化为更具语义结构的查询条件。例如，用户说“想买一台适合剪视频、预算八千左右的笔记本电脑”，模型可以自动从中提取预算、性能需求、用途场景等信息，进一步对接检索流程。与此同时，当缺乏用户历史行为时，LLM 还能通过语义推断生成合理的初步偏好解释，为缓解冷启动提供了新的技术可能 [4] [23]，使得推荐不再完全依赖已有交互记录。

然而，将 LLM 直接引入推荐并非没有代价。最典型的问题是幻觉生成：模型可能会编造不存在的商品型号，误述产品规格 [23]，甚至在品类属性之间做出错误归纳。在数码消费等高度依赖真实参数和可验证信息的场景中，这些错误往往会导致明显的决策风险 [11]，用户难以容忍。此外，LLM 给出的推荐理由虽然往往听起来流畅自然，但并不总能清晰映射回可信的数据来源，用户无法判断其逻辑是否基于真实商品信息还是模型的“想当然”，导致实际应用上难以完全依赖。

3. 现有研究的空白

综合现有工作，可以看到几个尚未被充分解决的问题。其一，许多方法仍停留在“大模型接入推荐流程”的宏观层面，缺乏对具体使用情境的细致设计，难以让领域知识真正融入推荐链路，导致在专业类场景（如数码、家电、户外装备等）表现不稳定。其二，跨品类的一致化路由机制仍不成熟：不同类别对商品属性的表达方式、用户关注点、约束条件差异显著，系统很难保持统一的理解与响应能力，导致推荐质量随品类波动。其三，缺少能够有效依托结构化数据的稳健机制，用以约束模型的生成边界、抑制幻觉，并确保每一次推荐都能基于真实、可验证的商品信息。

本文提出的架构正是围绕这些空白展开：通过在语义理解、品类路由和数据约束之间建立更紧密的联动机制，希望在真实使用场景中构建一个既能发挥大模型表达力，又具备可验证性与稳定性的推荐系统框架。

（二）多 Agent 系统研究

1. Agent 的基本概念与理论基础

在人工智能研究中，Agent 通常被描述为能够感知环境、自主作出判断并执行行动的软件实体 [15]。这一概念的核心在于 Agent 通过感知、推理与行动三个环节形成一个持续闭环：感知负责收集外部环境的状态变化；推理阶段整合内部模型与经验，对输入信息进行判断；执行环节将推理结果落实为具体动作，从而影响环境并进入下一轮循环。基于这一框架，研究者逐渐形成了不同的 Agent 设计思路。

反应式 Agent 强调对环境变化的快速响应, 通常基于预设规则或简单的刺激-反应机制, 在要求实时性高但情境较为明确的场景中表现稳定。相对而言, 慎思式 Agent 更倾向于先构建内部模型, 再通过规划、策略选择等步骤做出决策, 适用于任务复杂、目标多元的情境。这两类 Agent 在灵活性、复杂度以及可解释性上呈现不同取向, 也构成了后续多 Agent 系统设计的基本背景。

2. 多 Agent 系统的分布式协作机制

多 Agent 系统 (Multi-Agent System, MAS) 通过让多个具有自主能力的 Agent 协同工作, 为处理复杂、动态或跨领域任务提供了一种可扩展的解决方案 [21]。在这一框架下, 系统通常采用分布式任务拆分和角色划分的方式, 让不同 Agent 在相对独立的职责范围内协作完成整体任务。这样不仅提升了系统在规模扩张时的可扩展性, 也增强了面对局部故障或信息不确定性时的鲁棒性。

在众多协作模式中, 层级式结构尤为常见。上层的协调 Agent 负责理解任务、判断意图并进行路由, 而下层的领域 Agent 则依据专业知识处理更细粒度的问题。由于职责边界清晰, 这一架构便于集成与扩展 [17], 与本文采用的“MultiCategoryAgent + 品类专家 Agent”模式天然吻合。

随着大语言模型的发展, 面向多 Agent 协作的工程化框架也逐渐成熟。例如 LangChain、LangGraph 等系统通过路由链 (Router Chain)、计划-执行结构以及工具调用接口, 使开发者能够更方便地构建多 Agent 之间的信息流动和任务依赖。这些框架强调模块化与可组合性, 让不同 Agent 能够在统一协议下协同处理复杂任务, 为大型推荐系统的构建提供了实际可行的工程路径。

3. 多 Agent 架构在推荐系统中的应用潜力

近年来, 随着对话式推荐、个性化推荐与可解释推荐的发展, 多 Agent 架构开始在推荐系统领域展现出明显的优势。对话式场景中, 用户的完整需求往往在多次对话中慢慢形成, 多 Agent 系统中的对话 Agent、意图识别 Agent 和领域专家 Agent 能组建分工明确的协作链路, 使用户的语义表达能够被不断细化并映射到更清晰的商品偏好结构。

在跨品类推荐中, 传统单体模型往往难以同时处理数码、家电、户外装备等类别之间差异巨大的属性体系, 而多 Agent 架构能够让每个品类由相应的专家 Agent 负责处理, 使推荐逻辑更具一致性和可解释性。同时, 工具调用能力让 Agent 可以在对话过程中实时访问结构化数据库、知识图谱或外部 API, 从而有效弥补大模型在知识时效性与事实准确性上的不足。

一系列工作证明了多 Agent 在复杂情境下的潜力。Park 等人在 UIST 2023 提出的 Generative Agents 框架展示了具备记忆、反思与社交行为的类人 Agent 如何形成稳定协作 [13]; 而 MetaGPT [10] 通过角色分配与任务流水线的方式, 实现了更偏工程化的大规模协作能力。这些成果说明, 在需要多轮推理、跨领域知识整合或实时信息检索的任务中, 多 Agent 具备良好的适应性。这些特性为构建更稳健、可解释、可扩展的推荐系统提供了重要的参考方向, 也为本文提出的系统架构奠定了理论与工程基础。

(三) 可视化辅助的 LLM 人机交互

随着大语言模型逐渐被应用于推荐与决策支持场景, 单纯依赖文本生成的交互方式逐渐暴露出一定局限。一方面, 模型的推理过程对用户而言难以观察, 推荐结果缺乏透明度; 另一方面, 当系统返回多个候选方案时, 用户需要在不同属性之间进行比较, 也容易产生较高的认知负担。同时, 对于需求尚不明确的用户, 传统对话界面往往缺乏有效的引导机制。近年来的研究开始尝试将信息可视化方法引入 LLM 交互界面, 通过图形化方式辅助理解模型推理过程、比较推荐结

果，并在交互过程中提供适当引导。围绕这一思路，相关工作大体可以从三个方面展开：推理过程的解释性呈现、推荐结果的可视化比较以及交互过程中的视觉引导机制。

1. 推理过程的可解释性可视化

对于普通用户而言，大语言模型的推理过程通常难以直接观察，推荐结论往往以最终文本形式呈现。这种“黑盒式”输出在一定程度上影响了用户对系统结果的信任。可解释性人工智能 (Explainable AI, XAI) 研究正是试图缓解这一问题，其中一种常见思路是通过可视化方式呈现模型决策的依据或推理路径 [9]，使用户能够理解推荐结论形成的基本逻辑。

Wang 等人^[19]在对话推荐系统研究中发现，如果系统能够以结构化步骤展示推荐理由，例如“因用户提及‘轻巧’，过滤掉重量超过 1.5kg 的机型”，用户对系统的信任度指标 (Trust Score) 明显高于仅给出文本推荐的情况 [19]。Strobelt 等人^[18]在 CHI 会议提出的注意力权重热力图可视化工具，则通过标注模型在输入文本中的关注区域，使非专业用户也能够直观地看到模型关注的关键词位置 [18]，这为 LLM 推荐系统的解释性设计提供了新的思路。另一类研究尝试将思维链 (Chain-of-Thought, CoT) 推理过程进行图形化表达 [20]，例如将逐步推理转换为流程节点或关系结构，从而使模型的推理过程能够以更直观的方式呈现。这类可视化形式不仅有助于系统开发者调试模型，也为用户理解与监督模型行为提供了可能。

2. 多维推荐结果的可视化比较

在实际推荐场景中，系统往往会返回多款候选产品。用户需要在价格、性能、重量、续航等多个维度之间进行比较，如果仅通过文本描述呈现信息，往往难以快速形成整体判断。信息可视化研究表明，合适的图表形式能够降低用户^[2]在多属性信息处理过程中的认知负荷 (Cognitive Load)，并有助于提升决策效率 [2]。

Amar 等人^[2]在信息可视化研究中总结了用户分析多维数据时常见的认知任务，包括数值检索、条件筛选、趋势识别以及异常值判断等。这些任务为面向决策的图表设计提供了重要参考。在多属性比较场景中，** 极坐标雷达图 (Radar Chart) ** 常用于展示不同对象在多个指标上的整体表现，它能够在同一视图中呈现各产品在不同维度上的相对优势与劣势，从而帮助用户快速获得整体印象 [22]。相比之下，当分析重点集中在某一具体维度时，** 分组柱状图 (Grouped Bar Chart) ** 通常能够提供更清晰的数值对比效果 [2]。在对话式推荐系统中，这类图表如果与 LLM 生成的推荐理由结合使用，可以同时提供数据层面的客观比较与语言层面的解释说明，从而形成更加完整的决策支持信息结构。

3. 对话交互中的视觉引导机制

在许多实际使用情境中，用户在初次提出问题^[3]时往往并未形成明确需求。如果界面仅提供自由文本输入，用户可能难以准确表达自己的意图，进而导致多轮低效对话。为改善这一问题，人机交互 (HCI) 领域开始尝试在对话界面中加入可视化交互元素，通过轻量级的视觉提示帮助用户逐步明确需求。

Amershi 等人^[3]在 IUI 会议的研究指出，在对话界面中加入结构化视觉提示，例如约束条件标签或意图确认卡片，可以在交互过程中帮助用户逐步细化需求，从而减少不必要的对话轮次。Cai 等人^[8]提出的生成式 UI (Generative UI) 概念则进一步扩展了这一思路，即根据当前对话上下文动态生成不同的界面组件，例如筛选条件、参数滑块或结果卡片。这种动态界面形式在探索型信息检索任务中表现出较高的用户满意度和任务完成率。与此同时，界面视觉设计本身也会影响用户对系统的初始感知。例如“玻璃拟态 (Glassmorphism)”等现代界面风格以及平滑的过

渡动效，已被相关可用性研究证明能够在一定程度上降低用户接触 AI 系统时的心理门槛，并提升感知可用性（Perceived Usability）与整体交互体验 [5]。

三、需求分析与方法设计

（一）需求层面分析

1. 调研设计与数据来源

在设计智能商品推荐系统之前，有必要首先了解用户在电商选购过程中所面临的实际问题。为此，本文围绕数码电子产品选购体验开展了初步问卷调研，共回收有效问卷 20 份。调研内容主要关注用户在信息获取、产品比较以及决策判断等环节中的使用体验，从而为系统功能设计提供依据。

问卷结果显示，大多数用户在购买数码产品时往往需要参考多个信息渠道，包括电商平台、视频评测以及社交媒体内容等。由于这些信息来源分散在不同平台中，用户在实际决策过程中需要花费较多时间进行整理和比较。因此，通过调研数据对用户行为进行分析，可以更清晰地识别当前电商辅助决策工具存在的不足。

2. 用户决策痛点分析

调研结果表明，当前电商环境下用户在商品选择过程中主要面临三类问题。

首先，不同平台之间的信息呈现高度分散状态。约 85% 的受访者表示，在查询商品信息时通常需要同时参考京东、B 站以及小红书等多个平台，但由于缺乏统一的数据整合方式，用户往往需要手动整理参数并进行横向对比，这一过程增加了决策成本。

其次，网络评测内容普遍带有一定的主观倾向。约 80% 的受访者认为，许多评测文章或视频中往往包含个人偏好甚至商业推广因素，使得用户难以从中提取真正客观的产品信息。

此外，一些技术参数本身具有一定理解门槛。例如“传感器单像素尺寸”“阻抗”等专业术语，对于普通用户而言较难直观理解。约 40% 的受访者表示，希望系统能够通过图表等方式对关键参数进行可视化展示，从而帮助用户更直观地比较不同产品的性能差异。

3. 系统功能需求归纳

基于上述问题，可以从输入、处理与输出三个层面对系统功能需求进行归纳。

在输入层面，系统需要能够理解用户较为口语化的需求表达。例如“3000 元拍视频的相机”这一类描述，实际上同时包含预算范围、使用场景以及产品类型等信息。因此系统需要借助大语言模型（LLM）完成语义解析与槽位填充（Slot Filling），从自然语言中提取结构化参数。

在业务处理层面，用户更倾向于获得数量适中且质量较高的推荐结果。调研显示，大多数用户希望系统在一次交互中给出 3-5 款重点产品，并附带简要说明推荐理由。因此系统需要基于客观参数完成数据筛选，并生成具有解释性的推荐说明。

在输出层面，系统应提供可视化展示方式，以帮助用户理解不同产品之间的性能差异。同时考虑到数码产品之间逐渐形成生态化使用趋势，系统还应能够提供跨品类设备的搭配建议，即生态推荐（Eco-system Suggestion）。

综合来看，系统设计需要兼顾通用性、客观性以及良好的交互体验，并通过关系型数据库作为底层事实来源，以减少大模型在推荐过程中可能出现的幻觉问题。

（二）系统总体框架设计

1. 系统整体运行流程

在明确系统需求之后，需要进一步构建能够支撑上述功能的整体架构。本文设计的系统以用户输入自然语言请求为起点，通过多 Agent 协同机制完成任务处理。

用户输入首先由前置模块进行语义识别，可以通过关键词检测或调用 LLM 完成场景分类。当系统识别出具体商品类别后，中央调度模块会结合当前会话上下文，将任务路由至相应的领域专家 Agent。

领域 Agent 随后解析用户需求，从中提取预算范围、使用场景等关键信息，并通过对象关系映射（ORM）访问底层关系型数据库。系统先根据硬性参数进行筛选，再结合场景规则进行匹配，从而生成候选商品列表。

在确定候选产品之后，系统会调用 LLM 生成推荐理由，并同时各项评分数据传递至数据模块，用于生成可视化图表，例如雷达图或柱状对比图。最终结果以图文结合的形式呈现在前端界面中。

2. 四层模块架构设计

在软件实现层面，系统整体架构可以划分为四个相对独立的模块层级。

第一层为表象展示与会话层，主要负责前端界面展示以及用户会话管理。该层负责接收用户输入，并展示系统返回的推荐结果，同时动态加载图表组件与商品信息卡片。

第二层为路由与意图解析层，其主要任务是处理自然语言输入，并将用户需求转换为结构化参数。通过对大模型输出格式进行约束，可以保证业务层能够直接读取相关数据。

第三层为业务决策与领域代理层，该层负责执行核心推荐逻辑，包括场景匹配、评分计算以及候选产品排序等操作，同时生成最终的推荐说明。

最底层为实体映射与事实数据层，该层存储结构化商品参数数据，是整个系统的主要数据来源。目前数据库覆盖多个数码产品类别，并包含数千条参数记录，通过严格的数据结构设计可以有效降低模型产生幻觉的风险。

3. 动态领域扩展机制

为了提高系统的可扩展能力，架构中还引入了动态领域代理机制。当系统检测到用户需求属于尚未预置的新商品类别时，可以调用 LLM 对该类别的评价维度进行初步推断，并临时构建相应的评分体系。

在这种情况下，系统会自动切换至咨询模式，以保证用户仍然能够获得基础信息服务。通过这种方式，可以在不修改系统代码的情况下扩展系统支持的商品类别范围。

（三）本章小结

本章首先通过问卷调研分析了用户在电商选购过程中面临的主要问题，包括信息来源分散、评测内容主观性较强以及技术参数理解困难等。在此基础上，总结了系统在输入理解、数据筛选以及结果展示方面的主要需求。

随后，本文提出了一种四层结构的系统总体架构，包括表象展示层、路由解析层、领域决策层以及事实数据层。该架构通过结合大语言模型的语义理解能力与关系型数据库的结构化数据约束，实现了语义理解与客观数据筛选的结合，从而为后续章节中推荐算法与系统实现提供了整体框架基础。

四、核心算法设计与系统设计

(一) 用户需求语义解析与参数提取

在智能商品选购系统中，用户需求的准确理解是推荐流程的起点。现实场景下，消费者通常通过自然语言表达需求，而这些表达往往具有明显的口语化特征，例如“我想买一台不差钱、能打游戏还能做后期修图的电脑”。此类描述同时包含主观评价和多个使用场景，因此难以直接转换为数据库查询条件。

虽然大语言模型（Large Language Model, LLM）在语义理解方面具有显著优势，但在长文本推理过程中仍可能出现注意力漂移或语义扩散问题。如果缺乏约束机制，模型在解析需求时可能生成不稳定甚至虚构的参数，从而影响推荐系统的可靠性。

为此，系统设计了一种基于思维链（Chain of Thought, CoT）的提示工程方法，并结合结构化 JSON Schema 对模型输出进行约束。在单轮交互中，模型需要完成关键参数的结构化提取。首先，系统识别用户输入中的核心使用场景，并提取为字段 `usage`，用于匹配系统内部的场景知识库（Golden Sets）。其次，将带有主观含义的价格描述转换为可计算的预算参数，例如将“不差钱”等表达映射为系统允许的最高预算值 `max_price = 999999`。随后，模型根据用户需求推断其关注的性能维度，并生成排序字段 `sort_field`。最后，对用户需求进行简要语义归纳，生成字段 `summary` 作为推荐解释文本。

Listing 1: 通用意图解析系统提示词构建

```
1 def _parse_intent_generic(self, user_msg: str, category_name: str,
2   fields_desc: str) -> dict:
3     system_rules = f"""
4     你是一个{category_name}导购专家，负责从对话中提取结构化信息。
5
6     【待提取维度（用户均可缺省）】：
7     1. usage（用途）：使用场景关键词
8     2. budget_level（投入）：预算描述
9     3. max_price（预算）：数字，单位元，默认 100000（无限）
10    4. sort_field（排序字段）：基于用户需求选择最合适的评分字段（{fields_desc}）
11
12    5. summary（摘要）：解析出的核心诉求
13    6. product_type（类型）：显式要求的特定类型
14    7. brand（品牌）：用户明确指出想要购买的品牌名
15    8. owned_items（已购设备）：用户提到已经拥有的产品品类或型号
16
17    【推理规则】：
18    - 预算缺省默认为 20000。
19    - 如果用户提到“不差钱”、“旗舰”、“顶配”，max_price 设为 999999。
20
21    输出严格 JSON：{{"max_price": 数字, "sort_field": "字段名", "summary": "
    核心诉求", "usage": "场景关键词", "product_type": "...", "brand": "品
    牌", "owned_items": []}}
```

此外，为解决系统预设商品类别有限的问题，系统设计了动态品类补全机制（Dynamic Category Scoring System）。当核心识别模块 `CategoryDetector` 检测到用户需求涉及未在系统预定

义类别中的商品时，系统将进入知识推理模式，由模型自动生成该品类的评价维度。例如在厨房电器场景中，系统可能生成“功率”“容量”“温控能力”等评价指标，从而构建新的评分体系。该机制使系统能够在无需预先配置数据结构的情况下扩展新的商品类别。

Listing 2: 基于大语言模型的动态评分体系构建

```

1 def build_scoring_system(self, category_key: str, category_name: str) -> Dict
2 :
3     system_prompt = f"""
4     你是一个{category_name}领域专家。请为该品类设计一套评分体系和使用场景。
5
6     要求：
7     1. 设计 3-5 个核心评分维度（如性能、续航、便携性等）
8     2. 每个维度需要：中文名称、英文字段名(XXX_Score格式)、权重、描述
9     3. 设计 3-5 个典型使用场景及对应的推荐关键词
10
11     输出 JSON 格式：
12     {{
13         "dimensions": [
14             {{ "name": "性能", "field": "Performance_Score", "weight": 0.3, "
15               description": "..." }}
16         ],
17         "scenarios": {{
18             "gaming": {{ "keywords": ["游戏", "电竞"], "presets": ["推荐品牌关
19                           键词"] }}
20         }},
21         "default_sort_field": "主排序字段名"
22     }}
23 """
24 # 调用 LLM 生成并缓存动态体系

```

(二) 异构商品多维评分模型

在完成用户需求解析后，系统需要对数据库中的商品进行统一的性能评价。然而，不同类别电子产品的参数体系差异较大，例如手机关注摄像头和电池容量，而耳机更强调频响范围和降噪能力。因此，不同类别之间难以直接进行性能比较。

为解决该问题，系统构建了双层评价维度体系。第一层为通用评价维度(Universal Dimension)，用于描述所有商品共享的基础指标，例如价格水平和综合性价比(Value Score)。所有参与推荐排序的商品均需要具备该层评价数据，从而形成统一比较基准。

第二层为专有性能维度(Specific Dimension)，用于描述不同产品类别的核心性能指标。例如在相机代理模块 CameraAgent 中，系统建立了“便携性评分(Portability Score)”“弱光性能(LowLight Score)”和“视频能力(Video Score)”等指标；在显卡代理 GPUAgent 中，则使用“游戏性能(Gaming Score)”和“散热表现(Thermal Score)”等指标。

Listing 3: 异构商品专有评分维度定义

```

1 class CameraAgent(BaseDbAgent):
2     def get_specific_dimensions(self) -> List[ScoringDimension]:
3         return [

```

```

4         ScoringDimension("便携性", "Portability_Score", 0.3, "如重量体积"
5             ),
6         ScoringDimension("低光画质", "LowLight_Score", 0.4, "暗光表现"),
7         ScoringDimension("视频能力", "Video_Score", 0.3, "视频拍摄能力")
8     ]
9
10 class GPUAgent(BaseDbAgent):
11     def get_category_config(self) -> CategoryConfig:
12         return CategoryConfig(
13             # ...
14             scoring_dimensions=[
15                 ScoringDimension("游戏性能", "Gaming_Score", 0.35, "游戏帧率
16                     表现"),
17                 ScoringDimension("创作性能", "Creative_Score", 0.25, "渲染与
18                     AI加速"),
19                 ScoringDimension("功耗散热", "Thermal_Score", 0.2, "温度与噪
20                     音"),
21                 ScoringDimension("性价比", "Value_Score", 0.2, "综合性价比")
22             ],
23             # ...
24         )

```

由于商品数据来源多样,不同参数的量纲和格式存在差异,系统在数据处理中引入了多类型归一化策略 (Normalization Strategy)。对于布尔型参数,系统采用固定权值映射;对于离散枚举变量,根据性能等级设置分档权重;对于连续数值变量,则采用 Min-Max 归一化方法进行标准化处理。

归一化后,各类指标通过加权求和 (Weighted Sum) 方式计算综合得分,并统一映射至 0-100 的评分区间。该评分不仅用于推荐排序,也为前端可视化模块提供稳定的数据结构,使系统能够生成雷达图等多维性能展示图形。

(三) 双层场景化推荐引擎

在商品评分体系建立之后,推荐系统需要从大量商品中召回最符合用户需求的候选结果。如果仅依赖数值排序,往往难以体现数码产品选购中的经验知识;而完全依赖固定规则,又可能在预算约束较严格时出现无推荐结果的情况。

为提高系统稳定性,本研究设计了双层场景化推荐引擎。第一层为专家知识预设机制 (Golden Sets)。系统将常见消费场景与优质机型建立映射关系,例如“电竞游戏”“户外 Vlog 拍摄”“高保真音乐欣赏”等。当用户需求中的字段 `usage` 与这些场景匹配时,系统优先在对应候选机型集合中进行筛选,从而利用领域经验快速缩小搜索范围。

然而,由于市场价格变化或预算限制,预设机型可能被全部排除。为避免推荐结果为空,系统设计了降级检索机制 (Fallback Search)。当预设集合在预算过滤后剩余商品数量不足时,系统自动触发全库检索。此时推荐算法不再依赖场景规则,而是根据用户最关注的排序字段 `sort_field` 对数据库商品进行排序,并结合预算条件筛选新的候选商品。

最终,系统通过集合并集与去重策略整合两类结果,确保每次推荐均能输出 3 至 5 款不同商品,从而在保证推荐质量的同时提高系统鲁棒性。

Listing 4: 双层场景化推荐检索策略

```

1 def handle_chat_generic(self, user_msg, history, model_class, keyword_map):
2     # 1. 解析意图 (获取 max_price, sort_field, usage 等)
3     intent = self._parse_intent_generic(user_msg, config.name, fields_desc,
4                                         history)
5
6     # 2. 匹配预设场景 target_scenario
7     # ...
8
9     # 3. 第一层: 专家知识场景召回 (Golden Sets)
10    results = []
11    if target_scenario:
12        candidates = self._get_preset_products(target_scenario, model_class)
13        results = self._filter_and_sort(candidates, intent, model_class)[:3]
14
15    # 4. 第二层: 降级全库检索兜底 (Fallback Search)
16    if len(results) < 3:
17        exclude = [i.id for i in results]
18        more = self._fallback_search(intent, model_class, exclude)
19        results.extend(more)
20        results = results[:3]
21
22    return results

```

(四) 跨品类生态推荐算法

随着智能设备种类的增加,用户在购买单一设备时往往同时关注设备之间的协同体验。例如,在选择笔记本电脑时,用户可能还需要显示器、蓝牙音箱或无线鼠标等配套设备。因此,系统在核心推荐流程之外设计了跨品类生态推荐模块(Eco-system Suggestion)。

该模块首先构建若干设备生态主题集合,例如“移动办公生态”和“家庭影音娱乐生态”。每个主题包含一个核心设备类别以及若干关联设备类别。例如在移动办公场景中,笔记本电脑作为核心设备,而显示器和蓝牙扬声器等设备作为扩展组件。

在算法实现上,系统采用关键词权重评分机制对用户需求进行生态分类。例如与办公场景高度相关的关键词(如“编程”“出差”)被赋予较高权重,而“文档”等通用词汇则赋予较低权重。系统根据关键词累计得分判断用户所属的设备生态类别。

在确定生态类别后,系统结合用户已有设备列表 `owned_items` 进行过滤,以避免推荐重复设备。例如当用户已拥有显示器时,系统将优先推荐其他辅助设备。最终,系统通过语言模型生成简要推荐说明,并附加在主推荐结果之后,从而形成完整的设备组合建议。

Listing 5: 跨品类生态环境定义与关联规则

```

1 ECOSYSTEMS = {
2     "移动办公": {
3         "name_en": "mobile_office",
4         "description": "适合移动办公、远程协作的数码产品组合",
5         "core_categories": ["laptop"], # 核心设备
6         "related_categories": ["monitor", "bluetooth_speaker", "smartwatch"],
7         # 关联外设
8         "scenarios": ["办公", "会议", "出差", "远程工作"],
9     }
10 }

```



```
8     "keywords": {
9         "high": ["办公", "工作", "商务", "出差", "会议", "远程"], # 高权重关键词
10        "medium": ["文档", "表格", "演示", "ppt"] # 中权重关键词
11    }
12 },
13 "专业游戏": {
14     "name_en": "professional_gaming",
15     "core_categories": ["laptop", "gaming_console", "gpu"],
16     "related_categories": ["monitor", "headphone"],
17     "scenarios": ["3A游戏", "电竞", "主机游戏", "PC游戏"],
18     # ...
19 }
20 }
```

(五) 本章总结

本章围绕系统核心算法设计，对需求语义解析、商品评分模型、推荐引擎以及跨品类生态推荐等关键模块进行了系统分析。通过引入思维链提示工程与结构化约束机制，系统能够稳定地将用户自然语言需求转换为结构化参数。通过构建通用维度与专有维度相结合的评分体系，解决了不同类别商品难以统一评价的问题。

在推荐策略方面，系统提出了结合专家知识与降级检索的双层推荐架构，从而在保证推荐质量的同时提升系统稳定性。最后，通过跨品类生态推荐机制，系统在单一商品推荐的基础上扩展了设备组合建议能力，为后续系统实现与前端展示提供了重要支撑。

五、系统架构实现与代码落地

(一) 全栈基础架构与运行环境实现

在第四章完成系统总体架构设计之后，本章进一步从工程实现角度说明系统的具体落地过程。系统实现阶段的核心目标是在保证功能完整性的前提下，使系统具备良好的可维护性与扩展能力。因此，在技术选型和架构实现过程中，本研究重点考虑开发效率、系统稳定性以及后期扩展需求。

在整体技术架构方面，系统采用 Python 作为主要开发语言，并构建了一套轻量化的全栈应用环境。其中，前端界面与交互逻辑基于 Streamlit 框架实现。Streamlit 可以通过 Python 脚本直接构建 Web 界面，从而显著降低前端开发复杂度。同时，该框架提供了基于状态驱动的界面刷新机制，使系统能够在用户输入发生变化时自动更新页面内容。

在系统交互过程中，用户的对话记录以及部分运行状态信息通过 `st.session_state` 进行统一管理。

如下所示的代码片段展示了应用入口状态的初始化逻辑，该机制能够在多轮交互中保持数据一致性，从而保证推荐流程能够持续依据用户输入进行动态更新：

Listing 6: 系统状态初始化（片段，摘自 `app.py`）

```
1 # 初始化状态管理
2 if 'agent' not in st.session_state:
3     st.session_state.agent = MultiCategoryAgent()
```

```
4 if 'messages' not in st.session_state:
5     st.session_state.messages = []
6 if 'last_results' not in st.session_state:
7     st.session_state.last_results = None
```

在智能推理部分，系统通过标准 API 接口接入大语言模型服务。该接口遵循 OpenAI API 的调用规范，并通过配置文件统一管理模型参数，例如 `config.LLM_MODEL` 和自定义 `baseUrl` 等。具体实现见如下代码：

Listing 7: 大语言模型客户端初始化（片段，摘自 `base_agent.py`）

```
1 self.client = OpenAI(
2     api_key=config.DASHSCOPE_API_KEY,
3     base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
4 )
```

通过这种方式，模型调用逻辑与系统业务代码保持相对独立，当需要升级或替换底层模型时，只需修改配置文件即可完成迁移。

在数据可视化方面，系统采用 Plotly 作为主要图表绘制工具。Plotly 支持浏览器端交互式图表展示，能够实现动态缩放、悬停提示等功能，使用户可以更加直观地观察产品参数差异和评分结果。

系统的数据存储采用 SQLite 关系型数据库，并通过 SQLAlchemy ORM 框架进行管理。ORM 技术能够将数据库表结构映射为 Python 类对象，从而避免直接编写 SQL 语句所带来的安全隐患，同时也提高了代码的可读性和可维护性。

在项目结构组织方面，系统入口文件为 `app.py`，主要负责前端界面逻辑与会话控制；商品类别识别模块位于 `category_detector.py`；而各类商品推荐逻辑则集中在 `electronics_agents.py` 中实现。通过这样的模块划分，系统各组件之间保持较低耦合度，从而提升整体架构的清晰性。

（二）多 Agent 推荐模块实现

在实际电商场景中，不同类别商品的评价标准往往存在较大差异。例如，相机产品通常更加关注画质表现和视频能力，而手机产品则更强调性能与续航能力。如果所有推荐逻辑集中在同一模块中，不仅代码结构复杂，也不利于系统后期扩展。

为了解决这一问题，系统在实现阶段采用了面向对象编程思想，并构建了多 Agent 推荐模块。系统首先在 `base_agent.py` 文件中定义了抽象基类 `BaseProductAgent`，用于描述所有商品推荐代理的公共行为。该基类通过 `@abstractmethod` 声明若干必须由子类实现的方法，例如获取数据库模型类型以及返回评分维度信息等，其核心结构如下：

Listing 8: 推荐代理基类定义（片段，摘自 `base_agent.py`）

```
1 class BaseProductAgent(ABC):
2     @abstractmethod
3     def get_category_config(self) -> CategoryConfig:
4         """获取品类特有配置"""
5         pass
6
7     @abstractmethod
8     def get_specific_dimensions(self) -> List[ScoringDimension]:
9         """获取品类特有评分维度"""
```

```

10         pass
11
12     @abstractmethod
13     def get_model_class(self):
14         """获取 SQLAlchemy 模型类"""
15         pass

```

在此基础上，系统在具备数据处理能力的基类 `BaseDbAgent` 中实现了统一的推荐流程方法 `handle_chat_generic`。该方法采用模板方法模式，将完整推荐流程划分为以下核心代码呈现的多个步骤：

Listing 9: 推荐流程核心代码（片段，摘自 `electronics_agents.py`）

```

1 def handle_chat_generic(self, user_msg, history, model_class, keyword_map):
2     # 1. 意图解析：提取预算、评分字段等信息
3     intent = self._parse_intent_generic(user_msg, config.name, fields_desc,
4                                         history)
5
6     # ... 匹配需求对应的场景并处理补丁 ...
7
8     # 3. 获取预设商品的候选并应用过滤排序规则
9     candidates = self._get_preset_products(target_scenario, model_class)
10    results = self._filter_and_sort(candidates, intent, model_class)
11
12    # 4. 生成单独的推荐理由、以及基于核心维度的可视化图表
13    reasons = self._get_individual_reasons(results, user_msg, intent, history)
14
15    charts, analyses = self._generate_visualization(results, intent)
16
17    return reasons, charts, results, analyses

```

通过这种方式，不同商品类别在实现时只需关注自身特有的评分逻辑，而无需重复编写完整流程代码。

在具体实现中，系统定义了多个继承自基类的子类。例如 `CameraAgent` 主要负责相机类产品推荐，而 `PhoneAgent` 则用于手机产品推荐。这些子类通过重写方法定义各自的评价指标与专属设置，例如 `CameraAgent` 定义的代码如下：

Listing 10: 相机代理实现（片段，摘自 `electronics_agents.py`）

```

1 class CameraAgent(BaseDbAgent):
2     """相机推荐代理"""
3     def get_specific_dimensions(self) -> List[ScoringDimension]:
4         return [
5             ScoringDimension("便携性", "Portability_Score", 0.3, "如重量体积"),
6             ScoringDimension("低光画质", "LowLight_Score", 0.4, "暗光表现"),
7             ScoringDimension("视频能力", "Video_Score", 0.3, "视频拍摄能力")
8         ]

```

此处定义的维度直接映射到系统对相机进行综合打分的策略中，而手机类则会相似地关注“性能表现”和“续航能力”等指标并为其分配比重。

为了实现不同商品类别之间的统一调度，系统在 `multi_agent.py` 中实现了中央调度模块 `MultiCategoryAgent`。该模块负责接收通用查询并调度子代理，其核心逻辑如下：

Listing 11: 多品类代理调度（片段，摘自 `multi_agent.py`）

```
1 def handle_chat(self, user_msg: str, history=None):
2     # 1. 使用识别器识别品类
3     category_key, category_name = self.detector.detect_category(user_msg)
4
5     # 2. 获取具体代理实例
6     agent = self._agents.get(category_key)
7
8     # 3. 调用代理处理业务
9     reasons, charts, results, analyses = agent.handle_chat(user_msg, history)
10
11     return reasons, charts, results, analyses, eco_suggestion
```

通过这种方式，系统能够灵活支持多种商品类型，并且在需要扩展新类别时只需新增对应 Agent 并注册，无需修改总体处理链路。

（三）商品数据层实现与数据清洗

推荐系统的有效性在很大程度上依赖于底层数据质量。因此，在系统实现过程中，本研究构建了结构规范的商品数据库，并通过数据清洗保证数据的准确性和一致性。

系统数据库通过 SQLAlchemy ORM 框架进行管理，所有数据模型统一定义在 `models.py` 文件中。各类商品数据表通过继承 `Base` 类建立实体对象，例如 `Camera`、`Laptop` 等类分别对应数据库中的商品数据表结构。

设计数据模型时，每个类别都包含基础属性如品牌 (Brand)、型号 (Model) 和价格 (Price)。同时针对不同商品类别，系统还设计了专门的参数字段和评分维度，例如通过 SQLAlchemy 定义的 `Camera` 模型：

Listing 12: 数据模型定义（片段，摘自 `models.py`）

```
1 class Camera(Base):
2     __tablename__ = "cameras"
3     id = Column(Integer, primary_key=True, index=True)
4     Brand = Column(String)
5     Model = Column(String)
6     Price = Column(Float)
7     # ... 特定参数字段 ...
8     Total_megapixels = Column(Float)
9     Weight_g = Column(Float)
10    Supports_4K = Column(Boolean)
11    # ... 预置的评分结果 ...
12    Portability_Score = Column(Float)
13    LowLight_Score = Column(Float)
14    Video_Score = Column(Float)
```

如上所示，手机数据表也会有自己的电池容量 (Battery_mAh) 等指标，这些结构化数据与评分参数能够为推荐算法在生成策略时提供更加细粒度的参考信息。

当推荐模块需要获取候选商品时，只需通过 SQLAlchemy 提供的查询接口执行相关方法即可。例如在进行按预算全库兜底搜索时，操作如下：

Listing 13: 数据库查询过滤（片段，摘自 electronics_agents.py）

```
1 query = self.db.query(model_class).filter(  
2     model_class.Price <= intent.get('max_price', 20000)  
3 )  
4 # 基于特定字段进行降序排序  
5 if hasattr(model_class, sort_field):  
6     query = query.order_by(desc(getattr(model_class, sort_field)))  
7  
8 return query.limit(3).all()
```

ORM 框架会自动将这些链式调用操作转换为底层查询 SQL 语句，并返回对象结果，极大地保证了系统的安全性并避免了 SQL 注入风险。

在数据导入阶段，系统对原始数据进行了必要的清洗处理。原始商品数据主要来源于网络爬虫抓取的 CSV 文件，其中部分字段存在格式不统一或缺失情况。因此，系统通过批处理脚本对原始数据进行预处理，包括去除特殊字符、进行数值类型转换以及处理缺失值等操作。同时，对于部分文本属性（如“支持”“不支持”等），系统将其统一转换为布尔类型。经过清洗后的数据最终通过 SQLAlchemy 的 SessionLocal 接口导入数据库，从而形成结构规范的商品数据集。

（四） 用户界面与交互式可视化实现

为了提高推荐系统的可用性，本研究在界面设计中加入了可视化展示功能，使用户能够更加直观地理解产品之间的性能差异。如果仅以文本形式呈现大量参数信息，用户往往难以快速完成比较，因此可视化展示在系统中具有重要作用。

系统前端界面基于 Streamlit 的状态驱动机制实现。当推荐模块返回结果后，界面会根据 st.session_state 中存储的数据自动刷新显示内容，从而形成类似单页应用的交互体验。

在商品展示方面，系统设计了统一的商品卡片组件 render_product_card()。该组件通过封装的 HTML 和通过 CSS 实现玻璃拟态效果进行视觉呈现：

Listing 14: 产品卡片前端渲染代码（片段，摘自 app.py）

```
1 def render_product_card(product, reason=None):  
2     # ... 解析商品属性与动态加载核心规格 ...  
3     card_html = f"""  
4     <div class='product-card'>  
5         <div class='product-title'>{brand} {model}</div>  
6         <div class='product-price'><span class='currency'>¥</span> {int(price  
7             ):,}</div>  
8         {score_html}  
9         {spec_html_str}  
10        <div class='reason-box'>{reason_html}</div>  
11    </div>  
12    """  
13    st.markdown(card_html, unsafe_allow_html=True)
```

使用这样的内嵌组件方案，极大方便在 Streamlit 等响应式数据大屏中输出不同类别的定制化信息，使页面结构更具备视觉区分度和清晰感。

在数据分析部分，系统提供了两类主要图表用于展示产品性能差异。其中第一类为雷达图 (Radar Chart)，用于展示单个产品在多个评分维度上的综合表现。

Listing 15: 可视化图表呈现 (片段, 摘自 visualizer.py)

```
1 # 封装雷达图绘制
2 fig.add_trace(go.Scatterpolar(
3     r=values ,
4     theta=labels + labels[:1], # 闭合图形线条
5     fill='toself',
6     name=f"{p.Model}",
7     opacity=0.2
8 ))
9
10 # 分组柱状图应用
11 fig.update_layout(
12     title=dict(text="核心能力多维对比"),
13     barmode='group', # 实现多产品并排横向对比
14     yaxis=dict(title='评分', range=[0, 110])
15 )
```

如代码所示，系统利用 Scatterpolar 绘制雷达图帮助直观观察能力面积大小和偏靠程度。第二类图表则是指定 barmode='group' 参数的柱状多维对比，并支持交互式的悬停以显示具体数值进行横评产品比较。

通过商品卡片展示与交互式图表结合，系统不仅能够输出推荐结果，还能够向用户展示推荐依据，从而提升系统的可解释性和用户体验。

(五) 本章小结

本章从系统工程实现角度介绍了推荐系统的整体实现过程。首先说明了系统全栈技术环境的搭建，包括 Streamlit 前端框架、SQLite 数据库以及基于 API 的大语言模型接口。随后介绍了多 Agent 推荐模块的实现方式，并说明了面向对象设计在系统扩展性方面的优势。

在数据层方面，本章对数据库建模与数据清洗流程进行了说明，并构建了结构化商品数据库，为推荐模块提供稳定的数据来源。最后，通过商品卡片组件与交互式图表设计，实现了推荐结果的可视化展示。

通过上述实现过程可以看出，本文提出的推荐系统不仅在理论设计层面具有可行性，同时也能够在实际应用环境中得到有效实现。

六、 系统评估与真实场景化分析

本章的工作重心落在验证上——用自动化脚本和多维度功能测试来检验系统是否真的好用、逻辑是否真的严密。评估严格围绕项目源代码中内嵌的验证体系展开，重点考察三个方面：意图解析的准确度、推荐逻辑能否形成闭环，以及可视化输出的联动效果是否符合预期。

(一) 评估方案设计与测试基准

系统的可靠性需要分层次地去验证。本研究为各核心模块设计了对应的自动化测试脚本，将定性判断转化为可量化的评估结果。

验证工作部署于配备 16GB RAM 的本地工作站 (CPU 为 Intel i7 系列), 软件栈基于 Python 3.10 构建。推理层接入阿里云 DashScope 平台的 Qwen 系列模型 (走 OpenAI 兼容协议), 事实数据库层采用 SQLite 3。

验证标准从两个层面展开。组件级单元验证借助 `verify_multi_category.py` 脚本, 分别对 `BaseProductAgent` 基类、`CameraAgent` 专家代理、`CategoryDetector` 品类识别器和 `MultiCategoryAgent` 路由中心进行连通性与逻辑完整性测试; 全链路流转验证则由 `verify_agent_flow.py` 等脚本模拟真实对话, 追踪从用户输入到系统输出——涵盖推荐理由、Plotly 图表与数据分析文案——的完整链路是否畅通。

在性能基准方面, 回归测试设定了两条硬性要求: 对于“相机”“笔记本”等预设商品品类关键词, 识别器的准确率必须达到 100%; 在网络条件正常的情况下, 从意图解析到输出三张联动图表 (Radar、Bar、Multi-dim) 的端到端耗时应控制在 3 至 5 秒以内。

(二) 核心功能逻辑验证

为检验系统在极端条件下的健壮性, 本节结合项目内的压力测试脚本, 重点测试了两类边界场景。

第一类是基于降级检索的边界条件测试, 测试用例参考 `test_recommendation.py`: 输入极低预算 (如 2000 元), 但要求推荐高性能相机或笔记本。系统的处理路径如下: `electronics_agents.py` 中的 `_filter_and_sort` 方法首先对专家预设集执行硬性过滤, 若过滤后符合预算的产品不足三款, 则自动触发 `_fallback_search` 兜底逻辑, 依据 `intent` 中的 `max_price` 字段重新发起 SQLAlchemy 数据库查询。这一测试直接验证了双层推荐机制的容灾能力——系统在严苛约束下不会因“无匹配数据”而中断对话, 也不会凭空捏造型号, 而是退而求其次, 从全库中找出性价比最高的备选方案。

第二类是动态品类泛化能力验证, 测试对象为 `CategoryDetector` 面对非内置品类 (如“扫地机”) 时的表现。结果显示, 系统能准确识别出 `is_new: True` 标识并激活 `DynamicAgent` 的维度推演逻辑, 借助 LLM 的常识推理自发生成了适用于“扫地机”场景的评估维度 (如清扫能力、避障系数等)。这说明系统的品类处理能力并非只在预设范围内有效, 面对长尾需求同样具备相当的弹性。

(三) 系统效能评估分析

与传统关键词查询相比, 纯粹的参数过滤在面对模糊表达时往往力不从心。`_parse_intent_generic` 建立的意图映射层能在这类语境下准确抽取 `usage` (用途) 和 `summary` (诉求摘要), 将用户需求更精准地对齐至数据库中的性能向量轴, 这是结构化提取相比纯文本检索的核心优势所在。

可视化层面, `visualizer.py` 生成的动态图表在可解释性与信息密度两个维度上均表现突出。`_get_individual_reasons` 方法将晦涩的数值对比转化为易懂的自然语言描述, 让推荐结果“有据可查”; Plotly 引擎将多维参数归一化至 0-100 区间, 生成的雷达图使用户无需深究参数含义, 便可快速感知不同产品在便携性、性能、续航等维度上的相对位置。两者配合, 使系统的输出结果从“给出答案”进一步迈向“解释答案”。

(四) 从输入到图表的终局案例演示

本节选取三个典型测试用例, 完整呈现系统从意图捕捉到图表反馈的处理过程。

1. 案例 1：大学生的轻薄办公场景（笔记本电脑）

当用户输入”我是大学生……续航一定要好……看视频写论文”时，MultiCategoryAgent 将请求分发至 LaptopAgent，系统命中 light_office 关键字配置，将 Battery_Score 设为排序主轴，数据库最终返回 MacBook Air 与联想小新两款机型。输出的雷达图以”续航”和”便携”为长半轴，直观印证了推荐理由，使用户无需逐条比对参数即可建立选购判断。

2. 案例 2：极致性能的游戏发烧场景（全场景生态组合）

当用户输入”不差钱……顶级 3A 内容”时，系统识别”不差钱”标签后将 max_price 上限设为 999999，随即并发调用 GPUAgent 与 GamingConsoleAgent（逻辑参见 ecosystem_config.py）。最终输出展示了 4090 显卡与高端电竞显示器的联动数据，LLM 生成的文案围绕”专业游戏”场景给出了完整的生态闭环建议，将单品推荐延伸为场景化组合方案。

3. 案例 3：追求性价比的摄影入门场景（相机）

参照 test_recommendation.py 中的入门场景配置，系统对比了 5000 元价位段内索尼 ZV-E10 与尼康 Z fc 的视频能力（Video_Score）。雷达图清晰勾勒出 ZV-E10 在视频防抖与对焦上的优势。对于初次选购相机的用户而言，这样的视觉呈现要比一张参数对比表更容易做出判断，也更契合该系统降低用户决策门槛的设计初衷。

（五） 本章小结

通过项目内嵌的验证脚本，本章对系统进行了系统性评估。结果表明，Agent 协同架构在应对复杂消费意图、处理极端预算约束以及生成联动可视化反馈三个方面均表现良好。整体技术链路的闭环性符合预期，系统确实能够将繁琐的产品参数转化为直观的视觉决策依据。

七、 总结与展望

（一） 本文贡献与总结

电商平台的商品信息历来以参数为主要载体，用户在下单前往往需要自行消化大量技术指标，而这种信息组织方式与真实的购物决策逻辑之间存在相当大的落差。本文以此为出发点，提出并实现了一套基于大型语言模型与多智能体协同的对话式数码产品推荐系统，尝试在技术层面缩短这一落差。以下从四个方面对本文的主要工作加以归纳。

1. 提出面向消费场景的对话式推荐范式

传统电商平台习惯于将产品信息拆解为参数条目——处理器核心数、屏幕刷新率、传感器尺寸逐一陈列，用户须自行在”读懂参数”与”做出选择”之间完成跨越。对熟悉技术规格的用户而言，这种呈现方式尚可接受；对大多数普通消费者来说，却形成了一道隐性门槛。本文将大型语言模型的自然语言理解能力与多智能体的任务分发机制加以整合，构建了以多轮对话为主要交互形式的推荐路径。用户无须预先了解产品规格，只需在日常语言描述使用需求，系统便可在对话过程中逐步完成意图识别、方案筛选与结果解释。

2. 构建兼顾灵活性与可靠性的双层推荐架构

大型语言模型在理解模糊表达方面具有明显优势，但若直接以其生成具体商品推荐，”幻觉”问题便难以规避——模型可能依据训练数据中的片段印象，输出不存在或参数有误的产品信息。

本文对此采取了双层处理策略：前一层由大模型负责意图解析与结构化查询生成，后一层将查询结果交由本地 SQLAlchemy 数据库核验执行，确保推荐列表中的每条记录均有据可查，而非依赖模型推断。当标准查询路径无法覆盖用户意图时，回退检索引擎作为补充机制介入。这种分工方式保留了大模型在语义理解上的灵活性，同时将幻觉风险限定在推荐结果之外。

3. 实现跨品类协同推荐

多数推荐系统以单品类为设计边界，品类之间互不关联。然而消费者的实际采购需求往往并不孤立——刚入学的学生在选购笔记本时，多半同时需要耳机与移动电源；摄影爱好者在考虑升级机身的同时，镜头与存储方案的选配同样在决策范围之内。本文设计了 `MultiCategoryAgent` 路由模块与 `EcosystemConfig` 生态配置，赋予系统跨品类识别与联动推荐的能力。当用户的描述涉及“专业游戏”或“办公生产力”等场景语义时，系统可自动将相关品类纳入方案，将原本分散的选购环节整合为一次连贯的决策过程。

4. 以可视化手段增强推荐的可解释性

推荐结果能否获得用户信任，在很大程度上取决于用户是否理解推荐的依据。若系统只输出结论而不提供说明，用户便无从判断推荐是否符合自身偏好。本文在呈现层面引入了基于 Plotly 的多类型动态可视化组件：雷达图用于多维性能的直观对比，柱状图用于单项指标的横向比较，多维散点图则允许用户从价格、性能、口碑等多个维度同时审视候选产品。推荐理由以平实语言而非技术术语输出，与图形内容相互补充。降低决策成本，不只是筛选出合适的选项，还在于让用户看懂推荐的逻辑。

（二） 局限性与改进方向

系统设计本质上是约束条件下的取舍，本文的实现方案同样存在若干值得正视的局限。以下逐一说明，并提出后续改进的可能方向。

1. 商品数据的实时性不足

当前系统的商品数据存储在本地 SQLite 数据库，更新依赖离线爬取，无法与电商平台保持同步。在功能验证阶段，这一设计尚在可接受范围内；但若投入实际使用，信息滞后的问题便难以回避——促销期间价格可能在数小时内完成调整，库存状态的变化更是持续发生。用户若依据系统呈现的旧价格做出决策，而结账时发现价格已明显偏差，推荐的实用性将大打折扣。接入京东联盟等电商开放平台的官方 API 是较直接的改进路径，既可实现价格与库存的动态更新，也能减少对自行爬取的依赖，降低因平台策略调整引发的维护压力。

2. 复杂语义场景下的解析局限

意图解析是推荐流程的入口，其质量直接决定后续环节的可用性。`CategoryDetector` 在处理常规模糊表达时运行稳定，但在面对高度隐含前提的沟通方式时便会出现偏差。这类表达在日常对话中并不罕见，例如通过反讽传达偏好（“要那种不用一年就卡顿的手机”），或借助多重否定界定需求（“不要太贵但也别太便宜，拍照不用特别好，但也不能差”）。通用大模型对消费语境下的情感语义缺乏专项适配，面对上述情形时容易出现关键信息遗漏乃至方向性误判。引入经过细粒度情感分析微调的模型，是改善这一问题的可行思路，但需要结合具体数据集进行评估，并非一概而论地提升模型规模便能奏效。

3. 个体偏好的持续建模缺失

系统当前的推荐权重来自人工预设的场景规则——LaptopAgent 中，“学生场景”对轻薄与续航赋予较高权重，“专业创作场景”则侧重显示色域与处理性能。规则化方案逻辑清晰、易于审查，但只能反映群体层面的共性特征，对个体差异无能为力。用户对品牌的长期使用习惯、对外观设计的审美偏向，乃至对某类参数的特殊敏感，这些信息在当前架构中完全缺失。系统每次会话均从零开始，既不记录历史行为，也无法在多次交互中积累对特定用户的认知。在隐私保护前提下建立轻量级个人偏好模型，是系统走向实用化需要面对的实质性问题。

4. 后续改进方向

基于上述分析，本文提出以下三个改进思路供后续参考。

其一，引入检索增强生成（RAG）以补充软性评价信息。当前推荐依据以硬件参数为主，缺乏对实际使用体验的呈现。而在消费者的决策过程中，握持手感、系统流畅度、售后口碑等非参数信息对最终选择的影响往往不亚于跑分数据。将 Bilibili 评测视频文字稿、什么值得买上的深度评测及用户评价纳入外挂知识库，通过 RAG 机制在参数比较之外同步检索真实使用反馈，可使推荐结果在“数据维度”之外兼顾“体验维度”。

其二，建立用户操作的反馈闭环。用户的每一次操作——重新生成方案、修改预算、反复对比某两款产品——都隐含着对当前推荐结果的评价信号。若能将这些信号系统性地收集并用于模型校准，系统便有条件从固定规则逐步过渡到基于真实交互数据的个性化策略。RLHF 路径在技术层面已有相对成熟的参考实践，工程实施的难点在于如何设计合理的信号采集粒度与更新触发机制。

其三，将核心推荐引擎与 Streamlit 前端解耦。Streamlit 适合快速验证原型，但在移动适配与分发灵活性上存在局限。若将推荐引擎以标准 API 形式独立部署，则可在此基础上分别接入微信小程序与多设备客户端。小程序无需安装、触达成本低；多设备流转则允许用户在不同终端间延续同一次咨询——手机上提问、电脑上深入比较、平板上查看可视化结果，各端衔接而不中断。

本文所构建的系统，目前仍处于原型验证阶段。其意义主要在于提供了一条将大语言模型理解能力与数据库事实约束相结合的可行技术路径，并在可解释性呈现上做了一些探索性尝试。已识别的局限性，构成后续工作的自然起点。

参考文献

- [1] Gediminas Adomavicius and Alexander Tuzhilin. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6):734–749, 2005.
- [2] Robert Amar, James Eagan, and John Stasko. Low-level components of analytic activity in information visualization. In *IEEE Symposium on Information Visualization (InfoVis)*, pages 15–21, 2005.
- [3] Saleema Amershi, Maya Cakmak, W. Bradley Knox, and Todd Kulesza. Power to the people: The role of humans in interactive machine learning. *AI Magazine*, 35(4):105–120, 2014.
- [4] Kelong Bao, Jiawei Zhang, Yushuo Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. Tallrec: An effective and efficient tuning framework to align llms with recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2023. arXiv:2305.00447.
- [5] Amy L. Baylor and Tim Richman. Gender and expert–novice differences in the perception of pedagogical agent persona. *Computers in Human Behavior*, 18(2):213–234, 2002.
- [6] Robin Burke. Knowledge-based recommender systems. In *Encyclopedia of Library and Information Systems*. 2000. Preprint available online.
- [7] Robin Burke. Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4):331–370, 2002.
- [8] Carrie Cai et al. Generative user interfaces: Designing dynamic interfaces with large language models, 2024. Online technical report.
- [9] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Franco Turini, Fosca Giannotti, and Dino Pedreschi. A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5):93:1–93:42, 2018.
- [10] Shuyu Hong et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [11] Zhengyuan Ji et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023. Also available as arXiv:2309.05922.
- [12] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
- [13] Joon Sung Park, Joseph O’Brien, Carrie Cai, Meredith Ringel Morris, Percy Liang, and Michael Bernstein. Generative agents: Interactive simulacra of human behavior. In *UIST ’23: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- [14] Michael J. Pazzani and Daniel Billsus. Content-based recommendation systems. In Peter Brusilovsky, Alfred Kobsa, and Wolfgang Nejdl, editors, *The Adaptive Web*, pages 325–341. Springer, 2007.

- [15] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2020.
- [16] J. Ben Schafer, Dan Frankowski, Jon Herlocker, and Shilad Sen. Collaborative filtering recommender systems. In *The Adaptive Web*, pages 291–324. Springer, 2007.
- [17] Peter Stone and Manuela Veloso. Multiagent systems: A survey from a machine learning perspective. *Autonomous Robots*, 8(3):345–383, 2000.
- [18] Hendrik Strobelt, Sebastian Gehrmann, Hanspeter Pfister, and Alexander M. Rush. Lstmvis: A tool for visual analysis of hidden state dynamics in recurrent neural networks. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):667–676, 2018.
- [19] Hongwei Wang, Fuzheng Zhang, Jialin Wang, Miao Zhao, Wenjie Li, Xing Xie, and Minyi Guo. Ripplenet: Propagating user preferences on the knowledge graph for recommender systems. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 417–426, 2018.
- [20] Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [21] Michael Wooldridge. *An Introduction to MultiAgent Systems*. Wiley, 2 edition, 2009.
- [22] Sheng Zhang et al. A novel multivariate data visualization tool that improves radar chart. *BMC Medical Research Methodology*, 23(84), 2023.
- [23] Yushuo Zhang et al. Collaborative large language model for recommender systems. *arXiv preprint arXiv:2311.01343*, 2023.